

Dynamic Programming

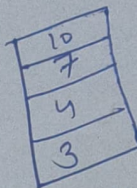
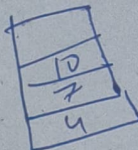
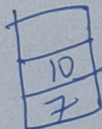
Heap

k^{th} Smallest Element

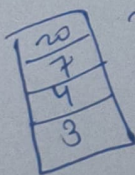
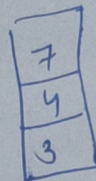
arr[]: 7 10 4 3 20 15 O/P: 7
k = 3 ↑ ↑ ↑
 3rd 2nd 1st

Easy → Sorting $n \log n$
Optimised → $n \log k$

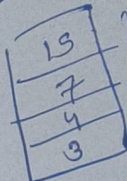
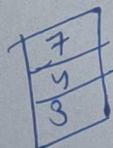
Smallest → max heap



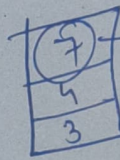
height > k
pop ✓



height > k
pop ✓



height > k
pop ✓



priority-queue<int> maxh;

for (int i = 0; i < size; i++) {

maxh.push(arr[i]);

if (maxh.size() > k) {

maxh.pop();

}

return maxh.top();

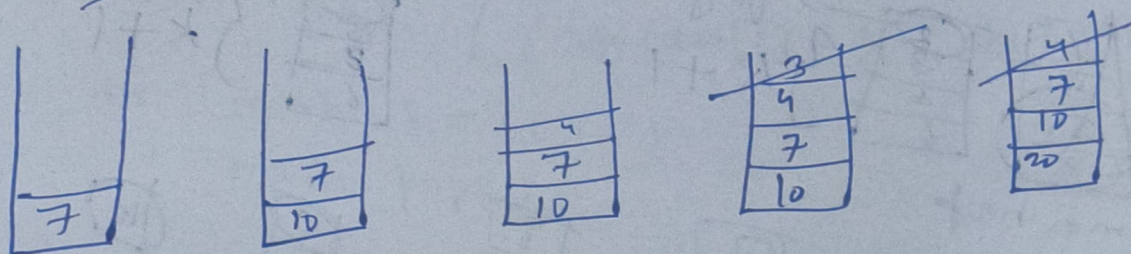
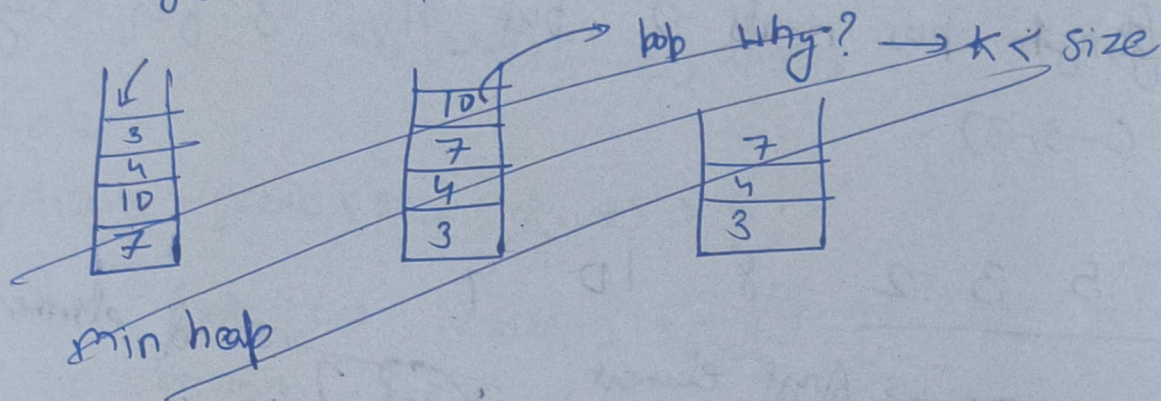
Return k largest Elements in an array

arr [] : 7 10 4 3 20 15

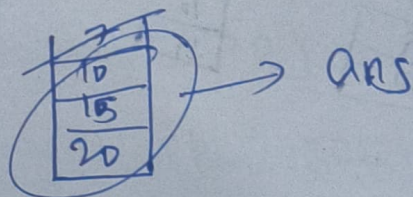
k : 3

o/p : 20 15 10

Largest $\rightarrow$ min



min heap



```

While (minh.size() > 0) {
    cout << minh.top() << " ";
    minh.pop();
}

```

3

Sort a k sorted array $\rightarrow$ nearly sorted

arr [] : 6 5 3 2 8 10 9

k : 3

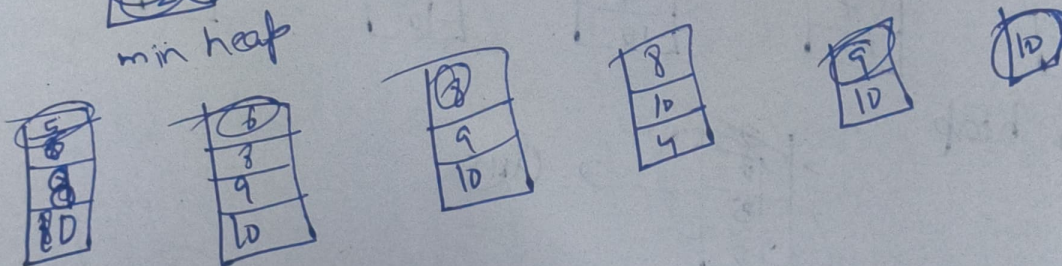
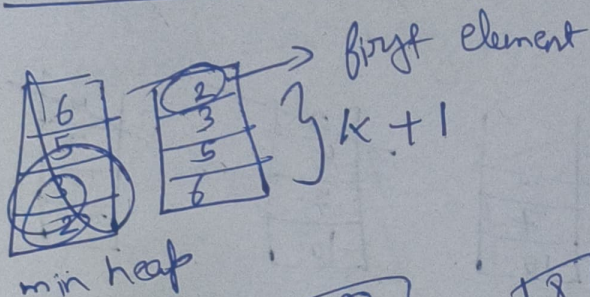
$\leftarrow K$ $\xrightarrow{K}$ ⑥ 5 3 2 8 10 9

~~The value's correct pos~~
max

The position's correct value is either k left or k right. $(i-k, i+k)$.

first pos needs 2 but its 3 for array $(-3, 3)$.

6 5 3 2 8 10 9



K Closest Numbers

arr[] : 5 6 7 8 9

$K=3$

$X=7$

Heap $\rightarrow$

pair $\{ \text{abs}(\text{arr}[i-k]), \text{arr}[i] \}$

Sol. ~~Brute force~~ $\rightarrow$

5 6 7 8 9
7 7 7 7 7

2 1 0 1 2
These

6 7 8

Closest $\rightarrow$ smallest $\rightarrow$ max heap

priority-queue $\langle \text{int} \rangle$ maxh;

$\langle \text{pair} \langle \text{int}, \text{int} \rangle \rangle$

$\text{abs}(\text{arr}[i] - x), \text{arr}[i]$

Code:

priority-queue $\langle \text{pair} \langle \text{int}, \text{int} \rangle \rangle$ p;

for (int i = 0; i < size; i++) {

maxh.push({abs(arr[i] - x), abs(i)});

if (maxh.size() > k)

maxh.pop();

}

while (maxh.size() > 0)

{ cout << maxh.top().second << " ";

maxh.pop();

}

Top K frequent elements

arr[]: 1 1 1 3 2 2 4

K = 2

map

1 $\rightarrow$ 3

3 $\rightarrow$ 1

2 $\rightarrow$ 2

4 $\rightarrow$ 1

Heap me insert

3, 1

1, 3

2, 2

1, 4


```
unordered_map<int, int> mp;
```

```
for (int i = 0; i < n; i++) {
```

```
    mp[arr[i]]++
```

```
}
```

```
for (auto i = mp.begin(); i != mp.end(); i++) {
```

```
    minh.push({i->second, i->first});
```

```
    if (minh.size() > k) minh.pop();
```

```
}
```

priority-queue<int, vector<int>, greater<int>> minh;

↓
pair<int, int>

Frequency sort

arr[]: 1 1 1 3 2 2 4

op: 1 1 1 2 2 3 4

3, 1
2, 2
2, 3
1, 4

```
while (maxh.size() > 0) {
```

```
    int freq = maxh.top().first;
```

```
    int ele = maxh.top().second;
```

```
    for (int i = 0; i <= freq; i++) {
```

```
        cout << ele
```

```
    }
    maxh.pop();
```

```
}
```


K-Closest Points to Origin

arr[] = $\begin{bmatrix} 1 & 3 \\ -2 & 2 \\ 5 & 8 \\ 0 & 1 \end{bmatrix}$ $K=2$

2D Array

sort Reverse them

Key based heap $\rightarrow \{ \langle \text{int}, \text{int} \rangle, \text{int} \}$
 $\downarrow \quad \quad \quad \downarrow$
point $\quad \quad \quad$ distance formula

pair<int, pair<int, int>> maxh; $\sqrt{x^2 + y^2}$ or $x^2 + y^2$

for (int i=0; i<n; i++) {

maxh.push({arr[i][0] * arr[i][0] + arr[i][1] * arr[i][1],
{arr[i][0], arr[i][1]}});

if (maxh.size() > k) {
maxh.pop();

}

while (maxh.size() > 0) {

pair<int, int> p = maxh.top().second;

cout << p.first << " " << p.second;

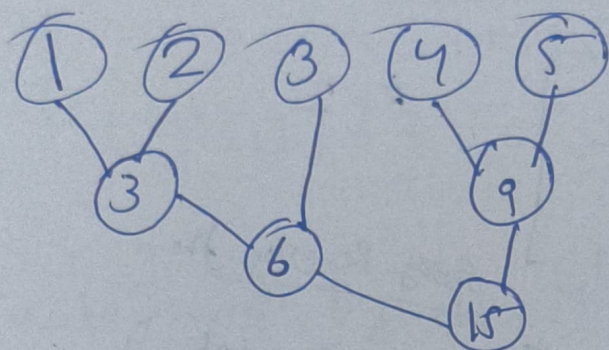
maxh.pop();

Connect Ropes

arr[] : 1 2 3 4 5

$\rightarrow$ optimal merge pattern

1. DP 2. Tree/graph 3. Bit



$$\frac{18}{3}$$

$$\begin{aligned} \text{Cost} &= 3 + 6 + 9 + 15 \\ &= 9 + 9 + 15 \\ &= 18 + 15 \\ &= 33 \end{aligned}$$

1. Add element to min heap
2. Take first two and do ~~calculation~~ ^{sum}
3. ~~Keep~~ Put back in heap
4. Repeat till only one element
5. Keep a variable for adding all these ~~costs~~

Code:

```
minh < >, cost = 0
while (minh.size() >= 2) {
    int first = minh.top();
    minh.pop();
    int second = minh.top();
    minh.pop();
    cost = cost + first + second;
    minh.push(first + second);
}
return cost.
```


Sum of Elements between K_1 smallest and K_2 smallest numbers

ex 1 3 12 5 15 11

$K_1 = 3$

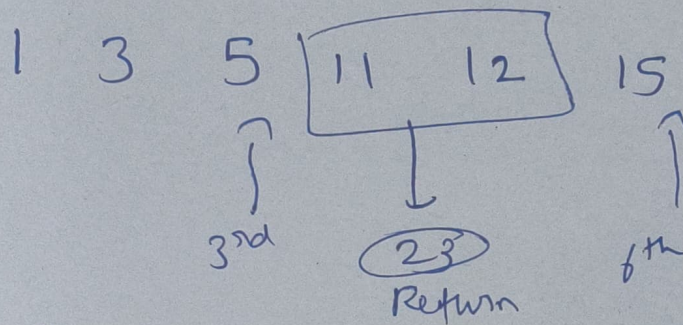
$K_2 = 6$

K_1^{th} smallest

K_2^{th} smallest

3rd smallest =

6th smallest =



First find 5 and 15.

Then traverse $\rightarrow$ greater than 5 and less than 15